Integrative Project Assignment
Logistics On Rails

sem3pi-2025-26 - version 2.21

October 30, 2025

Abstract

This document outlines the practical work to be developed within the Integrative Project for the 3rd Semester courses: ARQCP, BDDAD, ESINF, FSIAP, and LAPR3. The assignment involves the development of a computer solution to support the Logistic Operators on Railway Infrastructure. This document provides a brief introduction to the problem to be solved, as well as some fundamental concepts related to Logistics, Railway Networks, and Rolling Stock (including trains and freight wagons). Finally, the functional and non-functional requirements of the solution to be developed are presented, as well as the operational and technical details of the Integrative Project.

Table 1: Version History

Version	**Description**
1.0	Kick-off version
1.1	Minor errors corrected; Structure for repository proposed; User Stories (Sprint 1) for LAPR3 updated
2.0	Added User Stories (Sprint 2) for ARQCP, BDDAD, and ESINF
2.1	Minor punctuation errors corrected; ARQCP User Stories numbering corrected.
2.2	Added User Stories (Sprint 2) for LAPR3
2.21	Minor errors corrected

Contents

1 Integrative Project **3**
1.1 Team operation framework . . . 3

2 Description of the Problem **4**
2.1 Logistics in Railway Networks . . . 4
2.2 Railway System . . . 4
2.2.1 Railway Infrastructure . . . 5
2.2.2 Rolling stock (vehicles) . . . 6
2.2.3 Operations and management . . . 7
2.2.4 Routes, Paths and Freights . . . 7
2.2.5 Safety and security systems . . . 7
2.3 System Users . . . 7

3 Minimum Viable Product (MVP) **8**
3.1 Computer Architectures (ARQCP) . . . 8
3.1.1 Sprint 2 . . . 10
3.2 Databases (BDDAD) . . . 11
3.2.1 Sprint 1 . . . 12
3.2.2 Sprint 2 . . . 13
3.3 Information Structures (ESINF) . . . 14
3.3.1 Sprint 1 - Cargo Handling at a Railway Terminal . . . 14
3.3.2 Sprint 2 - Spatial & indexed queries . . . 21
3.4 Laboratory-Project 3 (LAPR3) . . . 23
3.4.1 Sprint 1 . . . 23
3.4.2 Sprint 2 . . . 24
3.5 Non-functional Requirements . . . 24

1 Integrative Project

In this project, students should be able to analyze, design, and implement a computer-based solution to support and address some of the critical aspects of managing operations related to Logistics in Railway Networks. This project serves as a proof-of-concept covering a set of relevant modules for logistics on rails, such as Warehouse Management, Freight Dispatching and Analysis, and Station Traffic Security.

In line with the best practices learned in the course, particularly in the subjects Computer Architecture (ARQCP), Databases (BDDAD), Information Structures (ESINF), Applied Physics (FSIAP), and Laboratory-Project III (LAPR3), an iterative and incremental process will be used. Therefore, an agile approach based on SCRUM will be applied in team management, structured in three sprints of four weeks each.

The solution to be developed should consist of a set of applications created in PL/SQL, Java, and C/Assembly, depending on the requirements. To enhance the solution's maintainability and align with software development best practices, the implementation must follow a Test-Driven Development (TDD) approach.

1.1 Team operation framework

Each team with LAPR3 students must nominate a Scrum Master for each Sprint (rotative role) and create a Scrum Board in the Organization *Departamento de Engenharia Informática*, following these guidelines:

- Only one team member (the assigned scrum master) creates the Project Board with the title "sem3-pi-25-26-gNNN-board in which the team number must replace NNN, including padding zeros;
- The board privacy must be "private";
- Access (writer) must be given to TP and PL teachers assigned to the team's class;
- The project board must include the following columns (lists): Backlog, To-Do, In Progress, Testing, Done, and Logbook (to register the Daily Meetings, the Retrospectives, and other team-related info).

All teams involved in the integrative project must create the project repository on GitHub in the Organization *Departamento de Engenharia Informática*, following these guidelines:

- Only one team member creates the repository, with the title sem3-pi-25-26-gNNN-repo based on the provided **template**;
- The repository access must be "private";
- The organisation base role must be set to "No access"
- All TP and PL teachers from the involved class courses supervising the project must be added to the repository with the "Reader" role.

While defining the repository structure, please consider the following proposal (see figure 1). Dividing the folders into sprints or UC is not recommended.

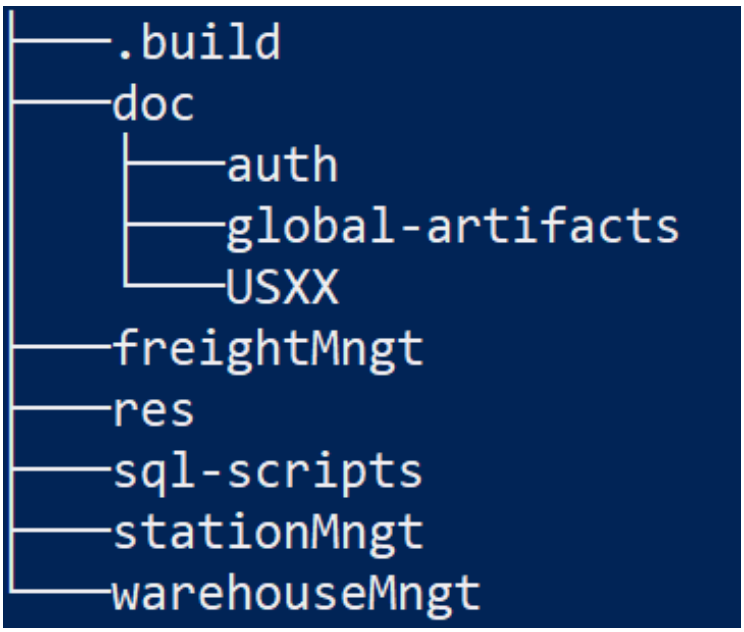


Figure 1: Proposed Repository Structure

2 Description of the Problem

Freight on rails is important because it is cost-effective, environmentally sustainable, reliable, and safe, which makes it essential for modern supply chains and sustainable economic growth. Currently, there are rail transport services that cover vast areas; for example, there are connections between Lisbon and Beijing for the transport of goods (this connection takes only 28 days to complete).

This section describes the problem to be solved: the development of a software solution that addresses some of the critical aspects of managing operations related to Logistics in Railway Networks.

2.1 Logistics in Railway Networks

Logistics of goods on railways refers to the management, planning, and execution of transporting freight by train, covering the entire supply chain process from origin to destination. It involves coordinating infrastructure, rolling stock (locomotives, wagons), scheduling, cargo handling, warehousing, and integration with other modes of transport (such as by road, sea, and air).

2.2 Railway System

A railway system for goods transportation (often called rail freight transport) is the network of trains, tracks, terminals, and supporting infrastructure dedicated to moving cargo rather than passengers (see figure 2 depicting Medway Portugal Map[1]). It plays a critical role in supply chains by allowing the efficient movement of bulk goods, raw materials, manufactured products, and containers between regions and countries.

[1] https://www.medway-iberia.com/pt/pages/mapas-de-rede

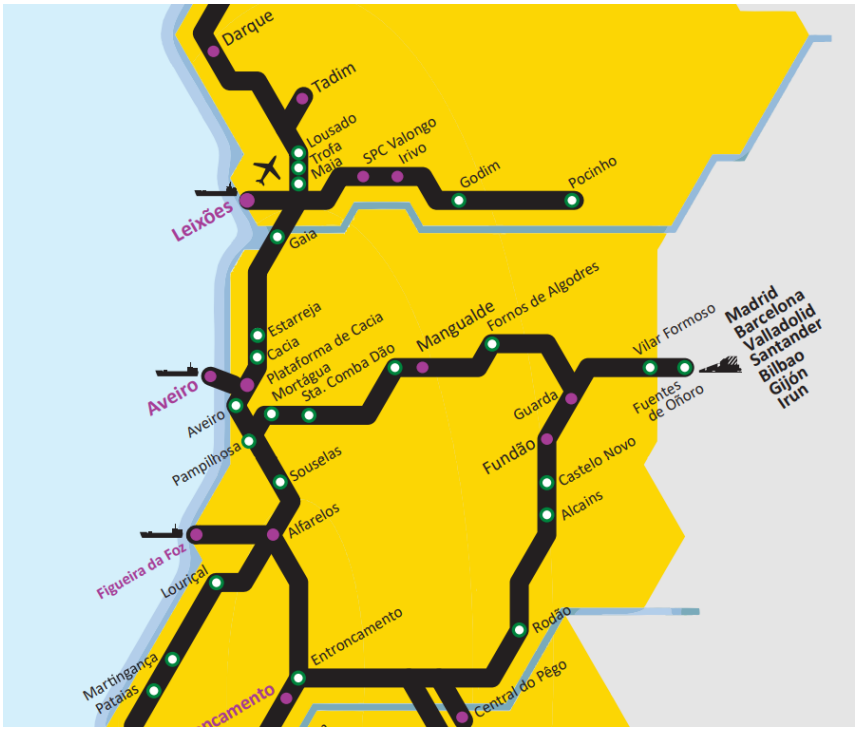


Figure 2: Excerpt of Medway Railroad Network

2.2.1 Railway Infrastructure

The railway infrastructure includes tracks, stations, bridges, and tunnels:

- Rail tracks and lines: The physical routes on which freight trains travel, built to connect stations and terminals, often connect industrial zones, ports, cities, and distribution centers. A line connecting two endpoints is composed of one or multiple line segments. Rail line segments can be single or double track. In the single-track case, trains must cross at stations or designated areas named sidings, whereas in the double-track case, trains can cross without losing speed and without requiring specific facilities. Line segments can also be electrified; in this case, they allow the circulation of electric locomotives. Important physical characteristics of line segments are the gauge (width), maximum weight (kg/m), and speed limit. A rail line has an owner, which is also responsible for the maintenance on all its segments, but the line may be used by trains of any operator.

- Freight yards, stations, and terminals: Facilities for loading, unloading, sorting, and assembling trains. Depending on their importance and purpose, facilities may include warehouses, refrigerated areas, and grain silos. Some terminals can be intermodal, where containers can be transferred between trains, trucks, and ships.

- Bridges, tunnels, and signaling systems: are engineering and safety elements that support long-distance freight movement, but they are not to be considered in the current problem.

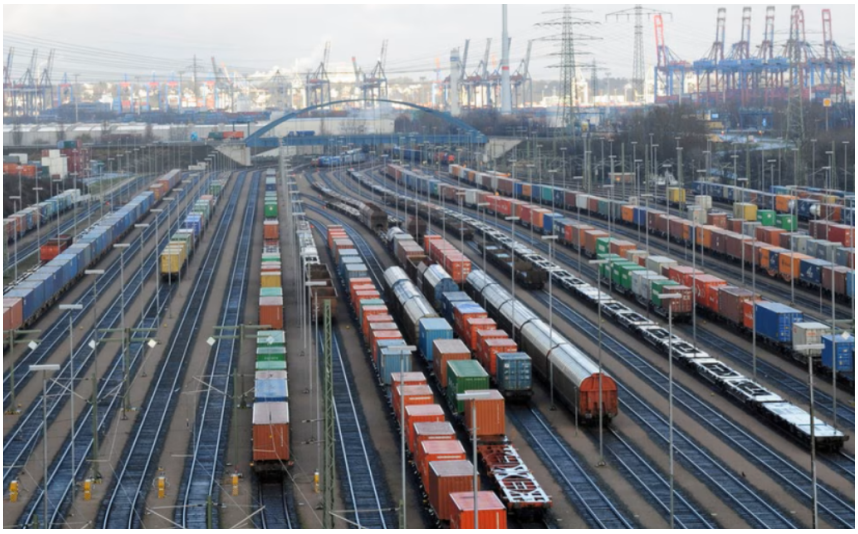


Figure 3: Hamburg Port Terminal

2.2.2 Rolling stock (vehicles)

Rolling stock includes all vehicles that move and use the railway infrastructure, such as locomotives, trains, and wagons. The rolling stock belongs to the train operators (train operating companies).

- Locomotives: Engines that pull the freight trains. Locomotives can be powered by electricity or diesel fuel. They can be coupled together in a train to provide the power required to pull all the freight carriages. Some of the most important characteristics are the make, model, year of entry into service, type (diesel, electric), power, and acceleration, dimensions, maximum/total weight, fuel capacity (only diesel engines), voltage (only electric engines), frequency (only electric engines), and gauge (some may support multiple gauges). The weight on the track can be calculated as the total weight/number of wheels on the tracks.



Figure 4: Freight Train with multiple locomotives

- Wagons/Freight cars: Designed for specific goods, such as: Boxcars (general goods), Flatcars (containers, vehicles, machinery), Tank cars (liquids, chemicals, fuel), Hopper cars (coal, grains, minerals), Refrigerated cars (perishable

goods). Besides the type of the wagon, payload, and volume capacity, dimensions, and tare are relevant characteristics. In Europe, 20, 40, and 45 ft containers are used, both regular and high cube. Staked containers are usually not supported in Europe, because electric lines and tunnels are quite common. Tank cars can be Chemical Tank Cars (Optimized for transporting chemicals with varying densities and high purity specifications, such as styrene and jet fuel), Mineral Oil Cars (Designed for the transport of various petrochemicals and mineral oils), and Pressure Gas Cars (used for liquefied gases like propane, butane, and other gaseous mixtures that require transport under pressure).

2.2.3 Operations and management

Some of the most important operations required in the freight railway system are:

- Scheduling and dispatching: Planning train movements to avoid delays and maximize efficiency.
- Loading and Unloading Operations: At terminals, depots, ports, or industrial hubs.
- Intermodal Transport: Combining rail with trucks/ships for 'door-to-door' logistics.
- Warehousing: Storage of cargoes in freight yards, terminals, and sidings.

2.2.4 Routes, Paths and Freights

A freight consists of a set of wagons that must be transported from endpoint A to endpoint B.

The path is the ordered list of stations through which the train must pass (not necessarily stopping) to travel from endpoint A to endpoint B. A route can be simple if it consists of a single freight or complex if it includes multiple freights. When a freight arrives at its destination, the wagons are decoupled from the train in order to be unloaded or loaded.

A train, when planned, has a scheduled route and a start date and time. As it runs, the instant it crosses or arrives at an endpoint is recorded. If the train stops at an endpoint, the leaving instant should also be recorded.

2.2.5 Safety and security systems

"still to be completed"

2.3 System Users

Below, the most common user profiles and respective functions are introduced:

- Train Drivers (Locomotive Pilots/Engineers): Operate trains, follow schedules, and ensure safe movement.
- Station Masters: Oversee station operations, coordinate arrivals and departures: ..... ;
- Storage Manager: .....;

- Freight Manager: .....
- Scheduler/Planner: Analyse, Design, and adjust train timetables to maximize efficiency.
- Terminal Operator:

3 Minimum Viable Product (MVP)

The project to be developed must consider the architecture presented in the diagram of the components in figure 5.

Figure 5: Architecture suggested for the solution to be developed

This project aims to develop a Minimum Viable Product, iteratively and incrementally; hence, the assignment is divided into three Sprints:

- Sprint 1 – weeks 3 to 6 – from 29/September to 26/October
- Sprint 2 – weeks 7 to 10 – from 27/October to 23/November
- Sprint 3 – weeks 11 to 14 – from 24/November to 4/January

A description of the MVP is provided for each sprint. The teams must follow the provided User Stories (US) and consider their sequencing and respective dependencies. By the end of each sprint, each team should be able to meet the specified requirements. The teams should be able to add the US to the Backlog, estimate them appropriately, and distribute them among team members. For easier reading, the US are separated by Course Unit (Subject).

3.1 Computer Architectures (ARQCP)

The purpose of the ARQCP component is to develop a system for managing a railway station. For that, consider the following:

- There are two kinds of users: System administrator and station operator.
- In each station, there is at least one track and no more than 99.
- A train can be in transit or stopped at a station.
- The station operator coordinates the train traffic at the station.
  - The station operator decides which track will be assigned to an arriving train or, if there is no available track, emits an emergency stop order.
  - The station operator gives the departure order to stop (stationary) trains.
- Each track is equipped with a light sign to indicate its state:
  - Green - track is free
  - Red - track is busy
  - Yellow - track is free, but assigned to an arriving train

  - Red (blinking) - track is inoperative

- The system administrator manages the station configuration as well as the station operation's data
- To avoid rejection, all station operator instructions should be timely logged.
- In each station, two sensors are present: temperature and humidity
- In each station, there is a synoptic board

The logical architecture of the Railway Station Management (stationMngt) system is presented in Fig. 6:

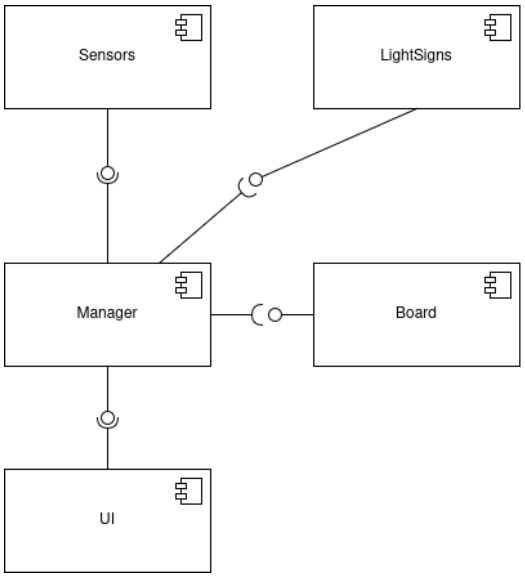


Figure 6: Logical Architecture.

- UI - The purpose of the UI component is to be the user interface of the StationMngt system. It is a console-based interface.
- Sensors - The Raspberry PI Pico is equipped with temperature and humidity sensors. This component provides the temperature and humidity at the railway station.
- LightSigns - The Raspberry PI Pico is equipped with LEDs. For each railway station track, there are LEDs to indicate each respective state. This component provides functions to manage the LEDs.
- Board - This component implements a console-based board. It provides all the functionalities for presenting data on a board, such as temperature, humidity, and the state of tracks, among others.
- Manager - The purpose of this component is to manage the railway station. It forwards the commands from the UI to the other components and manages all data (*e.g.,* users, log, tracks, sensors).

3.1.1 Sprint 2

The purpose of this sprint is to code (according to the RISC-V convention) a set of functions using **assembly**. These functions will be integrated into the software to be developed in Sprint 3.

- USAC01 - Implement the `int encrypt_data(char* in, int key, char *out)` function. It receives as input a string, `in`, and an integer, `key`, and for output it receives a string, `out`. The content of the input string is composed only of capital characters from the English alphabet (from A to Z). The value of `key` is in the range [1, 26].

  The function must encrypt the string `in` according to the Caesar Cipher and place the result into the output string `out`.

  It should return `1` if it succeeds, `0` otherwise (in this case, it should set `out` as an empty string).

- USAC02 - Implement the `int decrypt_data(char* in, int key, char *out)` function. It receives as input a string, `in`, and an integer, `key`, and as output, it receives a string, `out`. The content of the input string is composed only of capital characters from the English alphabet (From A to Z). The value of `key` is in the range [1, 26].

  The function must decrypt the string `in` according to the Caesar Cipher and place the result in the output string `out`.

  It should return `1` if it succeeds, `0` otherwise (in this case, it should set `out` as an empty string).

- USAC03 - Implement the `int extract_data(char* str, char* token, char* unit, int* value)` function.

  It receives as input two strings, `str` and `token`, and as output it receives two pointers, one for a string, `unit`, and another for an integer, `value`. The content of the `str` is formatted as follows:

  TOKEN`&unit:`*xxxxxxx*`&value:`*xx*`#`*TOKEN*`&unit:xxxxxxxx&value:xx`

  where *TOKEN* could be `TEMP` or `HUM` for temperature or humidity, respectively.

  This function extracts value and unit data from `str` according to the `token`. It should return `1` if it succeeds, `0` otherwise (in this case, it should set `value` to zero and `unit` to an empty string, `""`).

```
1 char str [] ="TEMP&unit:celsius&value:20#HUM&unit:percentage&value:80";
2 char token [] ="TEMP";
3 char unit[20];
4 int value;
5 int res =  extract_data(str,token, unit, &value);
6 printf("%d:%s,%d\n", res, unit, value); //1: celsius,20
7
8 char token2 [] ="AAA";
9 res =  extract_data(str,token2, unit, &value);
10 printf("%d:%s,%d\n", res, unit, value); //0: ,0
```

- USAC04 - Implement the `int format_command(char* op, int n, char *cmd)` function. It receives as input a string, `op`, and an integer, `n`, and as output, it receives a string, `cmd`. This function should format the output string as follows:

  - *CMD*`,x`, where *CMD* could be `RE`, `YE`,`GE` or `RB`, and `x` is two digit representation of `n` value.
  - *CMD* (`x` is ignored), where *CMD* could be `GTH` .

The *CMD* should be trimmed and capitalized, and it should return `1` if it succeeds, `0` otherwise.

```
int value = 5;
char str[] = "  rB  ";
char cmd[20];
int res = format_command(str, value, cmd);
printf("%d: %s\n",res, cmd); //1: RB,05

strcpy(str," Ye   ");
value = 25;
res = format_command(str, value, cmd);
printf("%d: %s\n",res, cmd); //1: YE,25

strcpy(str," Ye   ");
value = 125;
res = format_command(str, value, cmd);
printf("%d: %s\n",res, cmd); //0:

strcpy(str," aaa   ");
value = 25;
res = format_command(str, value, cmd);
printf("%d: %s\n",res, cmd); //0:

strcpy(str," gTh   ");
res = format_command(str, value, cmd);
printf("%d: %s\n",res, cmd); //1:GTH
```

A circular buffer is an array of constant length, and it is used to store data in a continuous loop. It is also known as a ring buffer because it stores the data circularly. The circular buffer has two pointers, one for the head of the buffer and another for the tail. The head pointer points to the location where we will insert the next element, and the tail pointer points to the location of the oldest element in the buffer. It is also required to know the length of the array as well as the number of elements in it.

- USAC05 - Implement the `int enqueue_value(int* buffer, int length, int *nelem, int* tail, int* head, int value)` function that insert the `value` into buffer. However, if the buffer is full, the oldest element should be removed to insert the new one. It should return `1` if after insertion the buffer is full, `0` otherwise.
- USAC06 - Implement the `int dequeue_value(int* buffer, int length, int *nelem, int* tail, int* head, int *value)` function which removes the oldest element from the buffer and outputs it in the `value` integer pointer. It should return `1` if it succeeds, `0` otherwise.
- USAC07 - Implement the `int move_n_to_array(int* buffer, int length, int *nelem, int *tail, int *head, int n, int* array)` function which removes the oldest `n` elements from buffer to `array`. It should return `1` if it succeeds, `0` otherwise (if there is no `n` elements to move).
- USAC08 - Implement the `int sort_array(int* vec, int length, char order)` function that sorts the array taking into account the `order` parameter, `1` one for ascending, `0` for descending. It should return `1` if it succeeds, `0` otherwise (if the `length` is less or equal to zero).
- USAC09 - Implement the `int median(int* vec, int length, int *me)` function which sorts the array and outputs the median value in the `me` parameter. It should return `1` if it succeeds, `0` otherwise (if `length` is less, or equal to zero).

3.2 Databases (BDDAD)

In this component, a database should be designed to model a railway system that supports the User Stories (US) listed below. Every team will have to implement

USBD01 to USBD04, and three of the remaining US. No team attending the same class can implement the same User Story (USBD05 to USBD20). The BDDAD PL class teacher will assign the US to each team of students.

3.2.1 Sprint 1

- USBD01 - This US was moved to LAPR3 requirements section.
- USBD02 - As a Product Owner, I want the relational model to be created (logical level).

  Acceptance Criteria

  - The data model should cover the requirements presented in sections 2.2.1 and 2.2.2 of the document, as well as any additional requirements resulting from the provided user stories.
  - Minimum expected requirement: a complete model in Visual Paradigm, including all connections and constraints, which allows the physical model script to be automatically generated in Visual Paradigm.
  - Minimum requirement above the expected: the presentation of a conceptual data model developed in Visual Paradigm, in addition to the minimum expected level.

- USBD03 - As a Product Owner, I want the relational model to be instantiated using the SQL DDL code generated by Visual Paradigm

  Acceptance Criteria

  - It will be shown on Oracle LiveSQL.

- USBD04 - As a Product Owner, I want to insert the provided data into the new database generated in USBD03.
- USBD05 - As a planner, I want a list of all line segments of the lines directly connecting two endpoints (stations, etc.).
- USBD06 - As a planner, I want a list of all line segments of a given gauge, including the line they belong to.
- USBD07 - As a planner, I want a list of all line segments of a given owner, including the line they belong to.
- USBD08 - As a planner, I want a list of all locomotives of a given type.
- USBD09 - As a planner, I want a list of all locomotives of a given gauge.
- USBD10 - As a planner, I want a list of all wagons of a given type.
- USBD11 - As a planner, I want a list of all wagons of a given gauge.
- USBD12 - As a planner, I want a list of all wagons of a given type and gauge that belong to a given operator.
- USBD13 - As a planner, I want a list of all locomotives of a given type and gauge that belong to a given operator.

- USBD14 - As a planner, I want to know the total number of kilometers of electrified rail tracks registered in the system.
- USBD15 - As a planner, I want to know how many lines depart from a given endpoint.
- USBD16 - As a planner, I want to know the minimum length of double-track segments in a given line.
- USBD17 - As a planner, I want to know the average length of the segments in a given line, by track type.
- USBD18 - As a planner, I want to know the number of segments in a given line, by track type.
- USBD19 - As a planner, I want to know the number of segments of a given owner, by track type.
- USBD20 - As a planner, I want to know the number of segments of each gauge size registered in the system.

  Acceptance Criteria for USBD05 to USBD20:

  - Minimum acceptance criteria: only the User Stories with data allowing their proper functioning will be evaluated.
  - Minimum expected requirement: demonstrated with the data imported from USBD04.

3.2.2 Sprint 2

Each team must implement USBD21 as well as 4 user stories from the set USBD22 to USBD30. Teams with 5 elements will have to implement 5 user stories. The user stories that each team will implement will be defined by the PL teacher.

- USBD21 - As a Product Owner, I want the relational model to be updated according to the changed requirements (logical level) and instantiated in Oracle DBMS.
- USBD22 - As a Product Owner, I want to insert the provided data into the updated database generated in USBD22.
- USBD23 - As a planner, I want a list of all line segments of a planned train route.
- USBD24 - As a planner, I want a list of all endpoints of a planned train route.
- USBD25 - As a planner, I want a list of all endpoints of a planned train route where the train does not stop.
- USBD26 - As a planner, I want a list of the wagons that were not used in any train in a given period.
- USBD27 - As a planner, I want a list of grain wagons that were used in every train that used grain wagons, in a given period.

- USBD28 - As a planner, I want a list of all locomotives of a given operator that support multiple gauges.
- USBD29 - As a planner, I want to know the percentage of electric locomotives of a given operator.
- USBD30 - As a planner, I want a list of all facilities that have grain silos, but no warehouses.

Acceptance Criteria for USBD23 to USBD30:

- Only user stories with data allowing their proper functioning will be evaluated. Every query must have at least one record.
- Whenever possible, data imported into USBD23 must be used.
- A PL/SQL function must be implemented for each user story, returning a SYS_REF cursor.

3.3 Information Structures (ESINF)

3.3.1 Sprint 1 - Cargo Handling at a Railway Terminal

This component aims to manage loading and unloading operations in a railway terminal. A railway terminal is a complex logistics node where freight wagons arrive and must be processed. The main areas of the terminal are:

- Circulation lines: tracks where wagons circulate to reach the parking areas or unloading docks.
- Parking lines: sidings where wagons wait temporarily before being sent to another destination.
- Unloading docks: access points to warehouses, where goods are unloaded for longer storage.

Warehouses connected to docks are organized in several aisles, each with several bays that can store a limited number of product boxes. Figure 7 presents a warehouse plant.

Train wagons transport several boxes of products. Each box is a small "lot fragment" of one Stock Keeping Unit (SKU) with quantity and an optional expiryDate, if it is a perishable product. When wagons arrive at the railway dock, depending on their load, they are dispatched into various bay warehouses. The following data sets provide the foundation for all functionalities, from inventory control to routing and replenishment.

- `bays.csv`: warehouseId, aisle, bay, capacityBoxes
- `wagons.csv`: wagonId, boxId, SKU, qty, expiryDate, receivedAt (ISO timestamp).
- `items.csv`: SKU, name, category, unit, volume, unitWeight.
- `orders.csv`: orderId, dueDate, priority.

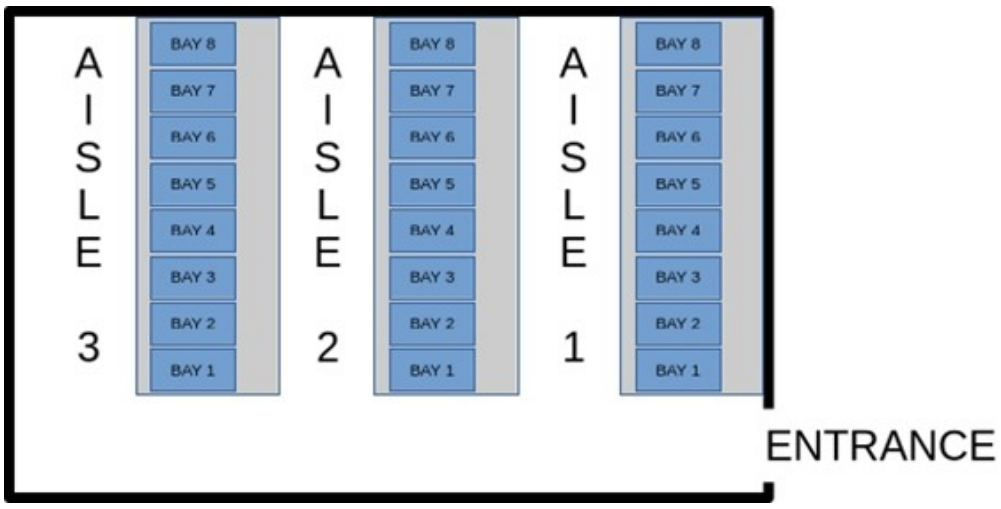


Figure 7: Warehouse plant

- `order_lines.csv`: orderId, lineNo, sku, qty.
- `returns.csv`: returnId, SKU, qty, reason, timestamp, expiryDate.

Each file must be validated during import. The system should reject invalid records and report the problem clearly, while still loading valid ones. Validation must include:

- Detection of unknown SKUs or wagons not linked to any known product.
- Rejection of negative or invalid quantities.
- Rejection of missing or invalid dates/timestamps.
- Ensure that the boxId is unique in the warehouse.
- If receivedAt is missing, reject.

Whenever an error is detected, the system must produce clear and deterministic messages, allowing users to identify the exact problem. For example, if a wagon record refers to a SKU not listed in items.csv, the error should explicitly identify the SKU and wagon in question. After successful import, all valid records must be stored in appropriate in-memory data structures, which will serve as the basis for all subsequent user stories (USEI01–USEI06). From this point on, algorithms will operate only on these loaded datasets, without directly accessing CSV files.

- USEI01 - Wagons Unloading (Inventory replenishment)

  Context: When wagons arrive at the terminal and are unloaded into warehouses, the sequence in which products are stored and dispatched is critical for maintaining inventory accuracy, traceability, and operational efficiency.

  Perishable goods must be handled strictly according to their expiry dates, following the First-Expired-First-Out (FEFO) principle. This ensures that goods closest to expiration are dispatched first, minimizing spoilage and waste.

  Non-perishable goods, without expiry dates, should be managed under a First-In-First-Out (FIFO) discipline. This guarantees a fair rotation of stocks

and prevents inventory stagnation. Each item with a unique SKU is stored in boxes, each with a unique location code (Aisle/Bay) to facilitate picking and restocking. Every movement of goods (whether unloading, relocation between bays, transfer to another aisle or bay, or final dispatch) is meticulously recorded in a warehouse management system (WMS). This ensures:

1. Accurate stock tracking by SKU and storage location.
2. Full traceability of all cargo throughout the cargo's lifecycle in the terminal.

User Story: As a terminal operator, I want unloading operations of wagons to automatically store inventory using FEFO and/or FIFO logic, so that I can ensure the correct dispatch order and minimize product spoilage.

Acceptance Criteria

1. Stock initialization/unloading wagons - The file wagons.csv contains the available stock per SKU. Each wagon's contents should be assigned to an aisle/bay, and their products must be inserted into the correct position inside the bay accordingly to the following rules:
   (a) boxes are ordered by expiry date (earliest first; null last)
   (b) receivedAt (oldest first)
   (c) boxId ASC (tie-break)
2. Dispatch operation - The operation dispatch must always consume stock from the "front" of the bay list, guaranteeing FEFO/FIFO behaviour. If a bay becomes empty, it remains in the WMS (the bay still exists), but the box list is empty; only delete empty boxes, not bays. Partial dispatch across multiple bays must be supported: if the target bay runs out, continue in the next bay, ascending number, that holds that SKU.
3. Relocation - The operation relocation must update a box's warehouseId/aisle/bay only; do not re-sort its new bay if its expiryDate/receivedAt is unchanged except for inserting it into the new bay's FEFO position.

- USEI02 - Order Eligibility & Allocation

  Context: A warehouse planner must fulfill the orders it receives. To achieve this, he needs to verify from the inventory which order lines can be dispatched and, for those, allocate concrete quantities from boxes using FEFO rules. The outcome is a list of per-line statuses and a set of concrete allocation rows (per order,line,box) for dispatch planning.

  User Story: As a warehouse planner, when I receive open orders, I want the system to examine current inventory and allocate quantities from boxes in FEFO order, and produce per-line statuses (ELIGIBLE, PARTIAL, UNDISPATCHABLE) and a list of allocation rows with box and bay information.

  Acceptance Criteria: Orders are processed by: priority ASC, dueDate ASC, orderId ASC. Within an order, lines are processed by lineNo ASC (input order).

  For each line, allocation walks the SKU's boxes in FEFO/FIFO order.

For each visited bay, allocate take min(remainingQty, box.qtyAvailable), reduce remainingQty, and continue until the request is satisfied or boxes end. The available box quantity never goes below zero during planning.

The system supports two modes: flag eligibility strict, partial, the default is strict:

- strict: a line is ELIGIBLE only if its entire requested quantity is allocated; otherwise it is UNDISPATCHABLE (no allocations for that line are kept).
- partial: if 0 ¡ allocated ¡ requested, mark the line PARTIAL and keep the allocated portion; if allocated = 0, mark UNDISPATCHABLE.

Returns:

- a list of orders eligibility with: orderId, lineNo, sku, requestedQty, allocatedQty, status;
- output allocations (one row per allocation fragment): orderId, lineNo, sku, qty, boxId, aisle, bay.

- USEI03 - Picking Plans

  Context: To satisfy the list of orders elaborated in the previous USEI02, it is required to collect products from one or more points in the warehouse – a process named "picking". Pickers often use a trolley (or cart) to collect items, but all trolleys have a weight (kg) capacity that must be respected. Therefore, it is necessary to have a pick plan that dispatches orders, or parts of orders, that fit within the capacity of the trolley. So, pickers can safely transport without exceeding the trolley's capacity.

  In the warehouse, the following packing heuristics are used:

  - First Fit (FF): Place the item in the first available trolley, where it fits. The scan order should be the input order of the allocation rows.
  - First Fit Decreasing (FFD): Sort unpicked allocations from US02 by weight from largest to smallest and place the item in the first available batch where it fits.
  - Best Fit Decreasing (BFD): Sort all unpicked allocations from US02 by weight, from largest to smallest, and put the item into the batch that will result in the smallest amount of remaining unused capacity, i.e., the tightest fit.

  If a single order line does not fit in the remaining capacity, either:

  - Split it: put part in the current wave, log a "partial allocation," and carry the remainder to the next trolley; or
  - Defer it entirely: log "skipped due to capacity," and put the full line in the next trolley.

  User Story: As a planner, I want to pack the allocation rows produced by USEI02 into capacity-bounded trolleys, choosing one of the packing heuristics. So pickers can complete runs without overloading trolleys.

Acceptance Criteria: There is no limit on the number of trolleys; their capacity should be defined by the planner. The planner must dispatch the orders, executing any one of the three packing heuristics on the same set of order lines. For each heuristic, it must be displayed:

- The total number of trolleys necessary to dispatch the orders
- Utilisation of each trolley (usedWeight/capacityWeight, e.g., 85% full)
- For each trolley, provide the picking plan with all: order ID, line number, aisle, bay number, boxId, SKU, quantity.

- USEI04 - Pick Path Sequencing

  Context: The warehouses are organized in several aisles that are open spaces between rows of bays, that provide pathways to move goods and access bays — areas for storing products. A bay location is a coordinate pair (aisle * 3, bay). The entrance is at coordinate (0,0) on the main corridor.

  Geometry & Distance Rules: Horizontal spacing between adjacent aisles = 3 units. The definition of the distance function D to go from coordinate (a1, b1) to coordinate (a2, b2) is given by the following movement model:

  - Same aisle ($a1 == a2$) : $D = |b1 - b2|$
  - Different aisles ($a1 \neq a2$): must go down from b1 to the main corridor, move horizontally and then go up to b2: $D = b1 + |a1 - a2| * 3 + b2$

  Sequencing Strategies

  - Strategy A — Deterministic sweep.
    - Start by sorting the bays of boxes ascending by aisle.
    - Next, calculate the distance to pick the boxes in that order (using the distance function D).
  - Strategy B — Nearest-neighbour (greedy) using the 2D distance above.
    - Start by sorting the bays using the distance function D.
    - Next, calculate the distance to pick the boxes in that order (using the distance function D).

  Route Distance Formula

  Let the sequence of the coordinates of the n bays to pick be c1, c2, ..., cn (each ci = (ai,bi), ai being an aisle and bi being a bay), and start c0 = (0,0). The total distance is calculated based on the distance formula D as follows:

$$D_{Total} = \sum D(C_i, C_{i+1})$$

  Example

  Pick the boxes of the bays in the following order: $(1, 8), (2, 2), (3, 4)$

  Strategy A (Ascending)

  Sorted order: $(1, 8) \rightarrow (2, 2) \rightarrow (3, 4)$

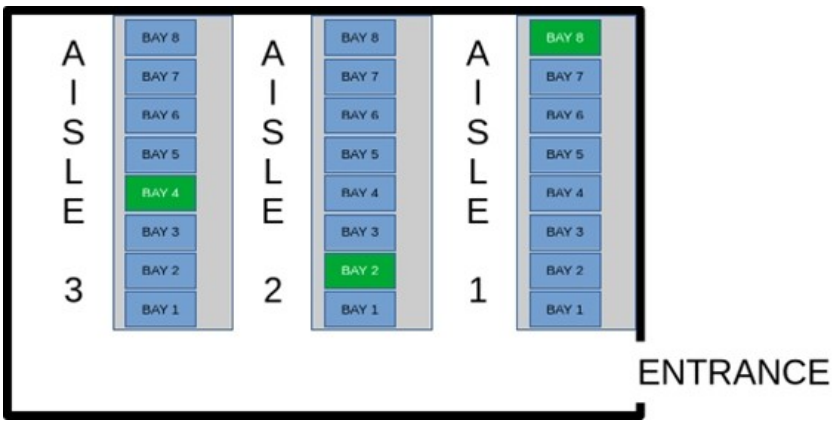


Figure 8: Warehouse Plant Problem

Distances:

$$(0,0) \rightarrow (1,8) : 0 + |0-1| * 3 + 8 = 11$$
$$(1,8) \rightarrow (2,2) : 8 + |1-2| * 3 + 2 = 13$$
$$(2,2) \rightarrow (3,4) : 2 + |2-3| * 3 + 4 = 9$$
$$Total = 33$$

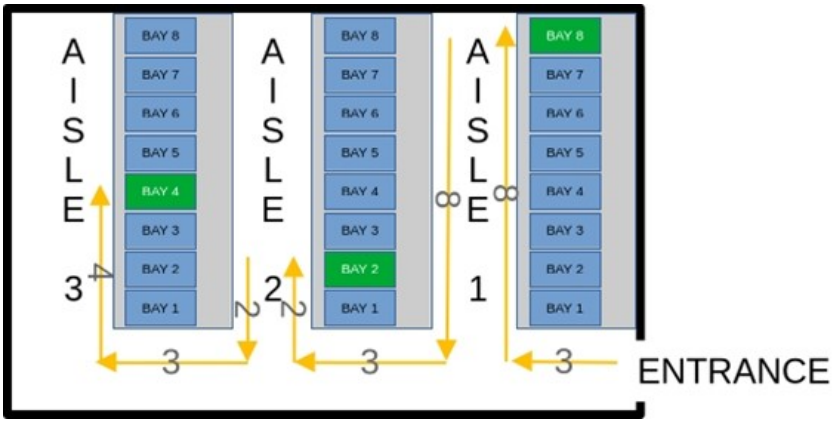


Figure 9: Strategy Ascending

Strategy B (Nearest-Neighbour) From the entrance, the distances are the following:

$$D((0,0),(1,8)) = 11$$
$$D((0,0),(2,2)) = 8$$
$$D((0,0),(3,4)) = 13$$

So the nearest neighbour to (0,0) is (2,2), which will be the first to be visited. Next, the nearest neighbour of (2,2) will be found:

$$D((2,2),(1,8)) = 13$$
$$D((2,2),(3,4)) = 9$$

The nearest neighbour to (2,2) is (3,4).

The final sorted order will be: (2,2) → (3,4) → (1,8)

$$(0,0) \to (2,2) : 0 + |0-2| * 3 + 2 = 8$$
$$(2,2) \to (3,4) : 2 + |2-3| * 3 + 4 = 9$$
$$(3,4) \to (1,8) : 4 + |3-1| * 3 + 8 = 18$$
$$Total = 35$$

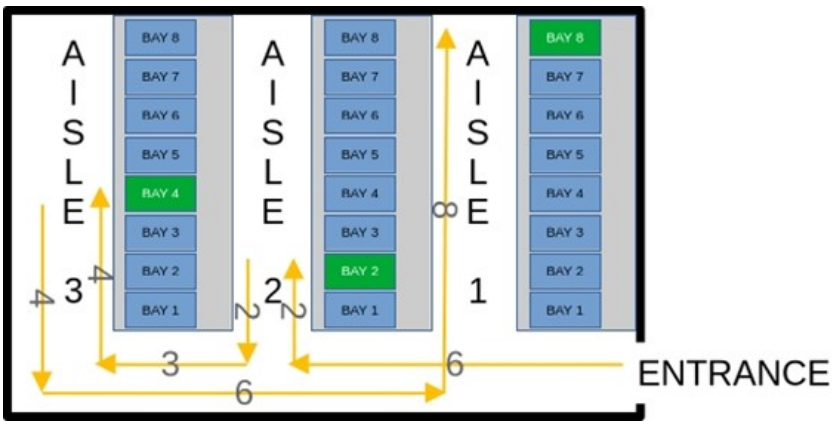


Figure 10: Strategy Nearest Neighbour

User Story

As a picker, I want to plan the task of picking up several boxes, sequenced using a 2D layout (aisles & bays), so I minimize walking distance starting from the entrance at the main corridor, with no return leg.

Acceptance Criteria

The input is the set of bays from the picking plan generated in the previous USEI03. Duplicate bays must be merged into a single stop (sum quantities) before sequencing; otherwise the same bay could appear multiple times in the path. For a picking plan (set of bays), both strategies must be executed, and for each strategy, return the path followed and the total distance.

USEI05 - Returns & Quarantine

Context: In a warehouse, returned products cannot immediately re-enter inventory. They must first be placed into quarantine to ensure quality and safety. This step prevents damaged or expired goods from being mixed with usable stock. Quarantined items are inspected in the reverse order they arrived (latest first), because these are usually the most urgent to process. After inspection, items may either be discarded if they are unsuitable or safely restocked back into inventory. To ensure accountability, every inspection action must be written into a log file with the relevant details.

Reasons for returns:

- **Customer remorse**: The customer returned the item even though it is still in good condition (e.g., unwanted gift, wrong choice). These items can often be restocked.

- **Damaged**: The product is physically broken or unusable. These items must be discarded.
- **Expired**: The product has passed its expiry date. These items cannot be sold and must be discarded.
- **Cycle count**: A mismatch was found during a stock audit. These items are not faulty but must be rechecked before restocking.

User Story: As a quality operator, I want returned goods to be placed in quarantine so that I can inspect them in the reverse order they arrived, latest first (descending by timestamp, ties by returnId ASC), process them, either discard or restock them, and create an audit file detailing the processing done to each product.

Acceptance Criteria

Handle customer returns putting them in quarantine first. Inspect each return; either discard or restock (as a new lot inserted into the inventory according to the FEFO/FIFO insertion rules defined in USEI01). For restock, create a new box with boxId="RET-"+returnId, receivedAt=now(), expiryDate as provided or null if unknown, and place it into the appropriate bay using the standard insertion rules. This ensures safe reintegration of good items back into the warehouse. If the return is discarded, it is flagged as no longer usable and will not be put back into stock.

Each inspection writes a line an external audit log file:

timestamp | returnId= . . . | sku=. . . | action=[Restocked|Discarded] | qty=. . .

(and, if partially restocked, include qtyRestocked and qtyDiscarded)."

For example:

```
2025-09-22 14:32 | returnId=R105 | sku=SKU123 | action=Restocked | qty=8
2025-09-22 14:40 | returnId=R106 | sku=SKU200 | action=Discarded
```

This log provides a clear history of all inspections and ensures that decisions can be reviewed later for auditing or troubleshooting purposes.

3.3.2 Sprint 2 - Spatial & indexed queries

In this sprint, teams will work with a real-world dataset of 64k (aprox.) European railway stations (station, latitude/longitude, country, timeZone, timeZoneGroup, and flags such as isCity, isMainStation, isAirport). The goal is to build efficient, deterministic query engines on top of these data using the BST/AVL implementations already made in classes, and to make a from scratch 2D tree for geographic queries.

- USEI06 - Time-Zone Index and Windowed Queries

  Context: Planners need fast, reproducible lookups of stations by time zone group, and convenient searches by station name and country without scanning the 64k-row CSV.

  User Story: As a planner, I want a set of BST/AVL trees that lets me quickly search by:

  - latitude

- longitude
- time zone group and country

So I can efficiently retrieve all stations of a time zone group by ascending order of country or in a window of time zone groups, such as ['CET', 'WET/GMT'].

Acceptance Criteria: Validate: non-empty name station, country, and time zone group; latitude ∈ [-90,90], longitude ∈ [-180,180]. Reject invalid rows. If multiple stations share the same coordinates, such as Lisbon Santa Apolónia and Lisbon Oriente, the tree must preserve all these stations sorted by name in ascending order.

Returns: 3–5 sample queries; Temporal analysis complexity.

- USEI07 - Build a balanced 2D-Tree on Latitude/Longitude

  Context: We need a spatial index over 64k (aprox.) European train stations to enable fast geographic queries. A KD-tree (k=2) on (latitude, longitude) supports range and nearest-neighbour operations much faster than scanning all stations.

  User Story: As a data engineer, I want to build a balanced 2D-tree using (lat,lon) so that range and proximity searches run efficiently and deterministically.

  Acceptance Criteria: The simplest way to ensure a balanced 2D-tree is to perform a bulk build; for this, develop a balanced build strategy that will use the BST/AVL of latitude and longitude created in USEI01 to efficiently construct the 2D-tree. If multiple stations share the same coordinates, such as 38.71387, -9.122271, which are linked to Lisbon Santa Apolónia and Lisbon Oriente, the 2D-tree must preserve all these stations sorted by name in ascending order.

  Returns: Size of the tree, its height, and all distinct bucket sizes; Temporal analysis complexity.

- USEI08 - Search by Geographical Area

  Context: Operations need to list all stations inside a map rectangle, optionally filtered by type or country. KD-tree pruning should avoid scanning the whole dataset.

  User Story: As a planner, I want to query the 2D-tree for all stations in a latitude/longitude range with optional filters, so I can extract relevant stations quickly.

  Acceptance Criteria: The region: lat ∈ [latMin, latMax], lon ∈ [lonMin, lonMax] (inclusive).

  The search should allow the optional application of filters with the following criteria:

  - isCity (true | false)
  - isMainStation (true | false)
  - country (PT | ES | all)

  Returns: 3–5 sample queries;Temporal analysis complexity.

- USEI09 - Proximity Search (Nearest-N with Filters)

  Context: Users need the N closest stations to a target coordinate (*e.g.,* for nearest major hubs). 2D-tree should minimize explored nodes.

  User Story: As an analyst, I want to find the nearest N stations to a given (lat, lon) using the 2D-tree, with optional filters, so I can get accurate nearby results efficiently.

  Acceptance Criteria: Use Haversine distance (km) with Earth radius between two (lat,lon) points. The search should enable optional filtering based on the time zone criteria.

  Returns: A list with n nearest neighbours; Temporal analysis complexity.

- USEI10 - Radius search and density summary

  Context: Operational questions often ask, "what stations lie within R km of a point?" and "how are they distributed by country and isCity?" We need both the list and a quick summary.

  User Story: As an operations planner, I want to fetch all stations within a radius R (km) of a target (lat, lon) using the 2D-tree, and get a summary by country and by isCity, so I can assess local coverage.

  Acceptance Criteria: Use Haversine distance (km) with Earth radius between two (lat,lon) points.

  Returns: A BST/AVL tree sorted by distance (ASC), and station name (DESC); Temporal analysis complexity.

3.4 Laboratory-Project 3 (LAPR3)

3.4.1 Sprint 1

- USLP01 - As a Product Owner, I want the domain model to be created (conceptual level). This model will be an essential communication element between all the stakeholders. The domain model diagram is a "dynamic" document, which should continuously reflect the stakeholders' shared understanding of the domain.

  Acceptance Criteria:

  - The data model should cover the activity of the railway system operation.
  - What is expected: a "basic" model shared in Visual Paradigm or the documentation in GitHub (*e.g.,* PlantUML).

- USLP02 - As a Station Storage Manager, I intend to access the Cargo Handling functionality at a Railway Terminal and the results through a user-friendly text-based interface.

- USLP03 - As a Traffic Dispatcher, I want to calculate the estimated travel time between two stations, with a direct connection between them, and a selected locomotive.

  Acceptance Criteria: In this version, acceleration can be assumed to be instantaneous. When presenting the results, the sections should be listed in the order in which they will be covered, and the speed should be considered in the calculation of the trip.

- USLP04 - As a Product Owner, I want a data dictionary/glossary to be created.

  Acceptance Criteria:

  - Glossary should be consistent with Conceptual Model Diagram
  - The main focus should be on the business concepts
  - The clarifications/definitions provided should be relevant for the problem interpretation and avoid generic definitions (Wikipedia-like definitions)

3.4.2 Sprint 2

- USLP05 - As a Product Owner, I want the conceptual domain model to be updated according to the changed requirements.
- USLP06 - As a Product Owner, I want a data dictionary/glossary to be updated according to the changed requirements.
- USLP07 - As Freight Manager, I want to use a scheduler that allows me to dispatch a list of trains. Each train has an associated route that can be simple or complex.

  Acceptance Criteria:

  - Each train/route has a planned date/time to depart;
  - At this stage, the route to be taken by each train is defined manually by the Freight Manager during the dispatch phase. The manual definition of the path is intended to guide the Freight Manager in situations in which the automatic system (to be developed) is not available.
  - The speed of travel should be calculated based on the limitations of the track, the combined power of the locomotives in the train, and the weight of wagons (which can be loaded or not).
  - The crossing of trains should be considered. In single lines, the crossing can be carried out when there are side lines or at the station before the crossing.
  - The scheduler must indicate the estimated times of passage for each train at the various points along the route and the necessary crossing operations.

3.5 Non-functional Requirements

This section defines some of the non-functional requirements that must be considered in the implementation of the project.

- The validation of business rules that must be followed during data registration and updating.
- The database will be the main repository of information for the system and must reflect the necessary data integrity. The information should be maintained in a remote database management system (DBMS).

- To enhance interoperability between existing or developing systems, the main application will be developed in Java. However, some components will need to be developed in other languages; the creation and management of the database will use PL/SQL, and the interaction with stations will be developed in C/Assembly. A significant part of the integration will be carried out through files.
- The class structure should be designed to allow easy maintenance and the addition of new functionalities, in line with best practices in object-oriented programming (OOP).
- During system development, the team must: (i) adopt best practices for identifying requirements and for OO software analysis and design; (ii) adopt recognized coding standards (e.g., CamelCase); (iii) use Javadoc to generate useful documentation for Java code.
- All the images/figures produced during the software development process should be recorded in SVG format.